

UTS namespace e hostname em um Docker-like system from scratch

Isolamento de identidade do host com control plane em Python, bridge em Cython e data plane em C++

Este documento investiga, em profundidade, como implementar suporte sólido a UTS namespace e hostname em um runtime de containers construído do zero. O foco está em separar corretamente identidade técnica, naming humano, hostname visível dentro do sandbox e aliases de rede, sem repetir os acoplamentos semânticos que tornam runtimes difíceis de evoluir.

Escopo: definição conceitual, leitura comparativa com Docker, proposta de System Design, Low Level Design, DDD, padrões de design, requisitos, testes e roadmap.

Pilha-alvo: Python no control plane e metadados; C++ no runtime e integração com kernel; Cython na fronteira tipada entre os dois.

Tese central: hostname não deve ser tratado como string solta do container, e sim como resultado de uma política de identidade aplicada a um sandbox proprietário de um UTS namespace.

Sumário

Resumo executivo 3

1. Introdução 3

2. Fundamentos conceituais 4

3. Como o Docker original trata UTS, hostname e naming 6

4. Arquitetura recomendada para Python, Cython e C++ 8

5. Proposta de System Design 8

6. Proposta de Low Level Design 11

7. Leitura em Domain Driven Design 15

8. Padrões de design relevantes 16

9. Requirements funcionais e não funcionais 17

10. Testes, observabilidade e operação 18

11. Roadmap de implementação 19

12. Riscos, trade-offs e anti-patterns 20

13. Conclusão 21

Referências 21

Resumo executivo

UTS namespace é o namespace Linux que isola dois atributos do kernel: o `hostname` e o `NIS domainname`. O ponto decisivo, porém, é que criar um novo UTS namespace não muda automaticamente o `hostname` percebido pela workload; o novo namespace nasce com uma cópia do valor do namespace de origem. Portanto, um engine que queira comportamento Docker-like precisa aplicar uma política ativa de naming e materializá-la via `sethostname()`, em vez de depender do efeito colateral da criação do namespace.[2][3]

Na prática, a feature exige quatro separações semânticas. A primeira é entre `container_id` e `display_name`. A segunda é entre esse nome humano e o `effective_hostname` efetivamente escrito no kernel do sandbox. A terceira é entre `hostname` e aliases de rede. A quarta é entre o valor vivo do kernel e suas projeções em user space, sobretudo `/etc/hostname` e `/etc/hosts`. Quando o modelo não faz essas separações, o runtime passa a misturar ergonomia de API, identidade da workload, resolução local e metadados internos em uma única string chamada `name`, o que cobra juros altos em restart, rename, exec, observabilidade e troubleshooting.[8][9][17]

Decisões arquiteturais recomendadas

ADR-01. O modo padrão deve ser `uts_mode = private`; `host` deve ser opt-in, auditável e governado por política.

ADR-02. O `hostname` padrão deve ser derivado de forma determinística a partir de um identificador técnico imutável do container, e não do nome humano exibido na CLI ou na API.

ADR-03. O runtime deve aplicar o `hostname` no kernel e também projetá-lo em `/etc/hostname`; o `inspect` deve mostrar o valor efetivo e sua origem.

ADR-04. O processo final da workload não deve receber `CAP_SYS_ADMIN` por padrão, de modo que a identidade definida no init permaneça praticamente congelada após o start.

ADR-05. A unidade proprietária do `hostname` deve ser um **Sandbox**, não um processo isolado nem o objeto de UX chamado “container”. Esse desenho prepara o motor para `exec`, restart e, no futuro, compartilhamento deliberado de namespaces.

O desenho recomendado para a pilha tecnológica é objetivo. Python concentra a linguagem de domínio, os metadados, as regras de negócio, a API, o `inspect` e a persistência. C++ concentra o launcher, a interação com `clone3()`, `setns()`, `sethostname()`, descritores de arquivo, sinais, `pidfds`, `bind mounts` e `execve()`. Cython cumpre o papel de anti-corruption layer tipada entre os dois lados, liberando o GIL durante chamadas bloqueantes e evitando que o processo do API server se torne o local onde namespaces são criados ou trocados.[5][6][7][13][14][15][16]

1. Introdução

Em runtimes de container, `hostname` costuma ser tratado como um detalhe menor de UX. Essa leitura é perigosa. Para uma enorme quantidade de workloads, o `hostname` ainda é um pedaço forte da identidade

percebida do processo. Ele aparece em prompts, agentes, logs, métricas, protocolos legados, scripts operacionais, middlewares de descoberta e ferramentas de observabilidade. Quando essa identidade vaza do host real para dentro de um container, o usuário percebe quebra de isolamento mesmo que filesystem, CPU e memória estejam razoavelmente compartimentados. Quando, na direção oposta, o runtime usa a mesma noção de “nome” para chave interna, alias humano, hostname e nome resolvível na rede, o sistema passa a carregar uma ambiguidade que complica cada evolução futura.

Este documento parte do contexto de um projeto que quer construir um Docker-like system from scratch com uma divisão de responsabilidades bastante sensata: control plane, metadados e API em Python; data plane, runtime e integração íntima com hardware e kernel em C++; Cython como cola tipada entre os dois. Dentro dessa arquitetura, a feature “UTS namespace e hostname” é pequena demais para justificar um subsistema gigantesco, mas importante demais para ser implementada como um punhado de flags jogadas em volta de `clone()`. Ela toca diretamente semântica de isolamento, consistência de naming, observabilidade, segurança e modelagem de domínio.[1][2]

A tese central deste texto é simples: **hostname deve ser modelado como resultado de uma política de identidade aplicada a um sandbox que possui um UTS namespace**. Isso muda o desenho inteiro para melhor. Fica claro por que o nome humano do control plane não deveria vaziar automaticamente para dentro do guest. Fica claro por que `/etc/hostname` e `/etc/hosts` são projeções de runtime, e não fontes primárias de verdade. Fica claro por que a fase correta para aplicar `sethostname()` é o init nativo, antes do `execve()` final. E fica claro por que a modelagem deve nascer preparada para *exec*, restart, rootless e eventual compartilhamento futuro de namespaces.[3][4][11][12]

2. Fundamentos conceituais

UTS namespace é o namespace Linux que isola hostname e NIS domainname. A documentação do kernel é explícita: mudanças feitas nesses campos são visíveis apenas para processos no mesmo UTS namespace. Outro detalhe igualmente importante é que, ao criar um novo UTS namespace, o kernel inicializa o novo namespace com cópias do hostname e do domainname vigentes no namespace de origem. Portanto, criar `CLONE_NEWUTS` não basta para “esconder” o nome do host do ponto de vista da workload. O motor precisa, deliberadamente, sobrescrever o valor copiado com a identidade que deseja apresentar ao guest.[2]

Essa observação resolve um mal-entendido frequente em projetos de runtime. Muitos engenheiros lembram que UTS namespace “isola hostname” e assumem que basta criar o namespace para que o container deixe de ver a identidade do host. Não é assim. O isolamento é potencial, não automático. O que o namespace isola é a possibilidade de alterações divergentes; ele não produz, por conta própria, uma política de naming. Essa política continua sendo responsabilidade do engine.[2][3]

Do ponto de vista de mecanismo, três famílias de syscalls são centrais. `clone()` e `clone3()` criam o processo já em novos namespaces quando usados com flags como `CLONE_NEWUTS`. `unshare()` move o processo chamador para namespaces recém-criados. `setns()` permite que um thread entre em um namespace já existente a partir de um descritor obtido, tipicamente, em `/proc/<pid>/ns/*`. Na prática de um engine, isso se traduz em três cenários distintos: criar um sandbox novo, entrar no namespace de um sandbox durante um *exec* e, em desenhos mais avançados, compartilhar intencionalmente um namespace entre múltiplas workloads.[1][4][5]

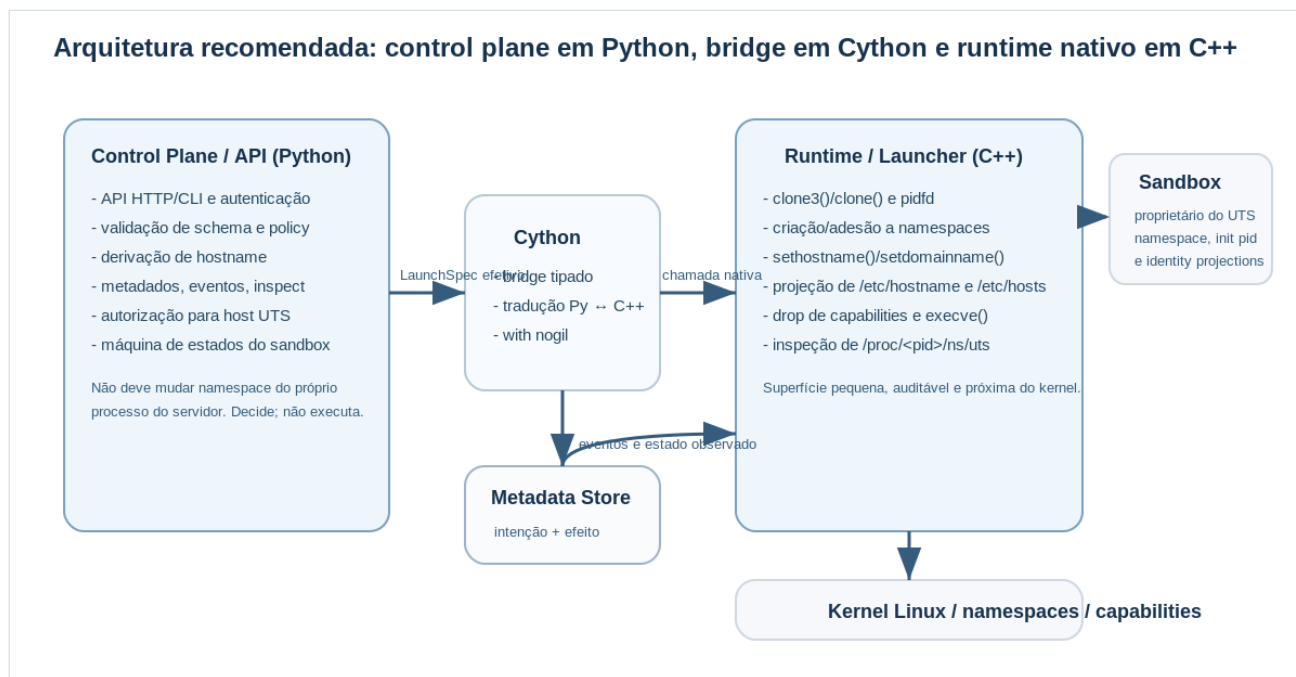
O syscall que materializa a mudança é `sethostname(2)`. O Linux exige que quem o chame possua `CAP_SYS_ADMIN` no user namespace associado ao UTS namespace do processo. A mesma documentação registra o limite `HOST_NAME_MAX = 64` no Linux. Essas duas informações são suficientes para orientar decisões de arquitetura. Primeiro, `hostname` precisa ser validado contra um limite de kernel, e não apenas contra convenções de UX ou RFCs de rede. Segundo, `rootless support` depende da ordem correta entre criação de user namespace, escrita dos mapeamentos de UID/GID e só então aplicação do `hostname`. Terceiro, o fato de `sethostname()` pedir `CAP_SYS_ADMIN` torna natural aplicar o `hostname` no init do runtime e remover a capability antes do processo final da workload assumir controle.[3][18]

Outro eixo conceitual decisivo é distinguir o `hostname` vivo do kernel de seus reflexos em user space. O arquivo `/etc/hostname` é uma configuração estática do espaço de usuário, não a fonte primária do valor transitório mantido pelo kernel. O manual `hostname(5)` trata explicitamente essa diferença. O mesmo vale para `/etc/hosts`, que é apenas uma tabela local de resolução e depende não só do `hostname`, mas também do contexto de rede, IPs atribuídos e aliases. Consequentemente, um engine limpo deve tratar ambos os arquivos como projeções de runtime que refletem uma decisão já tomada pelo control plane, e não como o repositório autoritativo da feature.[17][8]

Também convém eliminar uma confusão terminológica histórica: o `domainname` do UTS namespace refere-se ao NIS `domainname`, não ao “domínio DNS” no sentido moderno de `search domains` em `resolv.conf`. OCI e Docker preservam esse conceito por compatibilidade com a semântica do kernel, mas o motor não deveria apresentá-lo ao usuário como se fosse sinônimo de DNS `search suffix`. Isso é particularmente importante num design em que `networking` e `naming` serão bounded contexts distintos.[2][8][11]

Por fim, namespace não é apenas um estado implícito; ele é um recurso observável. O kernel expõe handles em `/proc/<pid>/ns`, e os números de device/inode dos links simbólicos permitem detectar se dois processos estão no mesmo namespace. Abrir um desses handles ou fazer `bind mount` sobre ele mantém o namespace vivo mesmo quando não há processos ativos. Esse detalhe é valioso para depuração, `exec`, inspeção e suporte futuro a sandboxes compartilhados.[1]

Figura 1. Arquitetura recomendada para separar control plane, Cython e runtime nativo



3. Como o Docker original trata UTS, hostname e naming

Docker é relevante aqui não por nostalgia, mas porque consolidou uma semântica operacional que muitos usuários já internalizaram. A primeira lição é que Docker não usa uma única noção de “nome”. Se o usuário não passa `--name`, o daemon gera automaticamente um nome legível e aleatório para a ergonomia da CLI. Em paralelo, a documentação de networking informa que o hostname padrão do container é o container ID e que esse valor pode ser alterado com `--hostname`. A mesma documentação distingue aliases de rede e explica que o container recebe seu próprio `/etc/hosts`; entradas customizadas do host não são herdadas automaticamente.[8][9]

Essa separação resolve um problema real. Nomes humanos são ótimos para UX, mas podem ser renomeados, podem conter caracteres inconvenientes para hostname e não são boas chaves técnicas. Hostname, por outro lado, deve respeitar limites de kernel, precisa ser estável do ponto de vista da workload e não deveria mudar porque o operador alterou um alias de conveniência na API. Ao desacoplar nome humano, ID técnico e hostname in-container, Docker evita que ergonomia de uso e invariantes de sistema se contaminem mutuamente.[3][8][9]

A segunda lição do Docker é sobre isolamento e compartilhamento. A CLI documenta explicitamente o modo `--uts=host` e também registra que ele não pode ser combinado com `--hostname` nem com `--domainname`. Essa restrição não é um capricho de UX; ela expressa uma verdade estrutural do kernel. Se o container compartilha o UTS namespace do host, o hostname deixou de ser um recurso privado daquele sandbox. Permitir configuração “independente” nesse cenário produziria uma API internamente contraditória.[9]

A terceira lição aparece quando conectamos Docker a OCI. A especificação OCI tem campos explícitos para `hostname` e `domainname`, e também modela namespaces Linux numa estrutura separada, com

possibilidade de indicar um `path` para aderir a um namespace existente. Isso é um bom padrão para um engine novo: o control plane pensa em termos de intenção de alto nível, mas o runtime recebe um spec efetivo que já sabe se o UTS será privado, herdado do host ou aderido a um handle existente.[11][12]

Há ainda uma quarta lição, menos óbvia, mas crítica para a implementação correta. O Docker não entrega `CAP_SYS_ADMIN` ao processo final por padrão; a documentação de privilégios e capabilities deixa claro que capabilities perigosas são removidas do conjunto padrão e que `--privileged` tem implicações de segurança severas. Como `sethostname()` exige exatamente essa capability no contexto apropriado, o caminho natural é aplicar o hostname ainda na fase de init do runtime. O processo final nasce vendo a identidade certa, mas não recebe, por padrão, o poder de alterá-la depois.[3][10]

A leitura correta, portanto, não é “Docker usa hostname”. A leitura correta é que o Docker opera com uma política consistente de identidade: nome de UX, hostname, aliases de rede e namespace mode são conceitos diferentes, governados por regras coerentes. Um runtime novo deveria preservar essa disciplina, mesmo que adote internamente um modelo mais rigoroso que o legado histórico do Docker.

Tabela 1. Eixos de identidade que o modelo do engine deve manter separados

Conceito	Escopo	Mutabilidade recomendada	Onde vive	Observação de design
<code>container_id</code>	Engine inteiro	Imutável	Metadata store, logs, auditoria	Chave primária técnica. Nunca deve depender de UX.
<code>display_name</code>	Control plane	Renomeável	API, CLI, eventos	Ótimo para ergonomia; ruim como fonte automática do hostname.
<code>effective_hostname</code>	Sandbox / UTS namespace	Imutável por política padrão	Kernel + inspect + projeções	Valor efetivo observado pela workload.
<code>requested_hostname</code>	API / domínio	Intenção de input	Metadata store	Nem sempre coincide com o valor efetivo; precisa ser preservado.
<code>domainname</code>	Sandbox / UTS namespace	Opcional	Kernel	Refere-se a NIS/YP, não a DNS search suffix.
<code>network_aliases</code>	Rede	Configurável	DNS embutido, network DB, /etc/hosts	Pertencem ao bounded context de networking.
<code>uts_namespace_handle</code>	Runtime	Derivado do estado	<code>/proc/<pid>/ns/uts</code> ou bind mount	Importante para exec, debug e futuros modos compartilhados.

4. Arquitetura recomendada para Python, Cython e C++

A divisão mais saudável para esta feature é uma arquitetura de **control plane em Python, runtime nativo em C++ e bridge tipada em Cython**. Python deve concentrar schema validation, regras de negócio, persistência, autorização, composição de responses e visão de domínio. C++ deve concentrar aquilo que exige proximidade com o kernel: `clone3()` ou fallback, sincronização parent-child, manipulação de descritores, `setns()`, `sethostname()`, bind mounts, drops de capabilities, supervisão por pidfd quando disponível e `execve()`. Cython deve ser tratado como uma camada pequena e precisa de adaptação entre as duas semânticas, sem renascer ali a lógica de negócio.[5][6][7][15][16]

Esse recorte é particularmente importante porque Python moderno, de fato, expõe APIs relevantes: `os.setns()` e `os.unshare()` existem no módulo `os`, e `socket.sethostname()` existe no módulo `socket`. Essas APIs são úteis para utilitários, protótipos e testes, mas isso não significa que o processo do API server deva manipular namespaces do próprio runtime. Mudar namespace de um thread de um processo long-lived e possivelmente multithreaded é uma decisão arquiteturalmente hostil: mistura control plane com data plane, aumenta a superfície de falha e torna o processo de aplicação um lugar ruim para lidar com sinais, file descriptors, sincronização de child init e rollback de estados intermediários.[4][13][14]

Dentro dessa arquitetura, o conceito decisivo é **Sandbox**. Mesmo que o MVP mantenha um sandbox para cada container, o modelo correto é pensar no sandbox como envelope proprietário dos namespaces, mounts de projeção, init pid, namespace handles e identidade efetiva. O container passa a representar a intenção de workload; o sandbox representa o ambiente de execução. Isso parece um detalhe de modelagem, mas muda profundamente a clareza do sistema: hostname deixa de ser tratado como propriedade de um processo individual e passa a ser corretamente entendido como propriedade do UTS namespace compartilhado pelos processos daquele sandbox.[1][2][12]

O contrato entre Python e C++ também merece disciplina. Em vez de mandar um dicionário amorfo e deixar o runtime reinterpretar regras, o control plane deveria produzir um **LaunchSpec efetivo**. Esse spec já deve carregar `sandbox_id`, `container_id`, `display_name`, `requested_hostname`, `effective_hostname`, `hostname_source`, `domainname`, `uts_mode`, `rootfs`, diretório de estado, política de capabilities e informações relevantes para rootless. A regra é clara: o runtime não deveria redescobrir regras de domínio; ele deveria executar um plano decidido pelo control plane.

No sentido inverso, o runtime deve devolver muito mais que um PID. O retorno mínimo útil inclui init pid, inode do UTS namespace observado, hostname efetivamente aplicado, estratégia de spawn utilizada e erros estruturados com estágio e `errno` quando algo falha. Sem isso, o `inspect` não consegue ser honesto e a equipe operacional não consegue responder onde o start falhou: se foi na criação do processo, no mapeamento de user namespace, no `sethostname()`, na projeção de arquivos ou já no `execve()`.

5. Proposta de System Design

5.1. Linguagem ubíqua

O primeiro artefato de system design, aqui, não é um diagrama de classes. É um vocabulário estável. O sistema precisa usar termos inequívocos: `container_id` para identidade técnica imutável; `display_name` para

alias humano de UX; *requested_hostname* para a intenção declarada pelo usuário; *effective_hostname* para o valor realmente escrito no kernel; *hostname_source* para dizer se esse valor veio de input explícito, derivação por policy ou herança do host; *uts_mode* para o modo de vínculo com o namespace; e *network_alias* para descoberta via rede. Sem essa disciplina, a palavra “name” passa a significar coisas diferentes em cada camada e o sistema fica semanticamente instável.

5.2. Política padrão e defaults

A política padrão recomendada é a seguinte: cada sandbox nasce com `uts_mode = private`; se o usuário não informa hostname, o control plane deriva um hostname estável a partir de um identificador técnico imutável do container; e o resultado é persistido no momento da criação, não recalculado a cada start. Esse desenho é superior a derivar o hostname do nome humano, porque nomes humanos podem conter espaços, acentos, maiúsculas, underscores, podem ser renomeados e respondem a uma política de unicidade diferente da identidade técnica do runtime. A melhor analogia operacional é a do Docker: nome amigável para o operador; hostname técnico e previsível para a workload.[8][9]

Convém adotar uma política de validação conservadora. Em vez de tentar ser permissivo demais, o engine pode escolher um modo padrão estrito que trate o hostname como rótulo ASCII minúsculo, sem acoplá-lo automaticamente a FQDNs, e que sempre aplique o limite de 64 bytes do Linux antes de delegar ao runtime. Em ambientes internos isso reduz ambiguidades entre nodename, hostname, FQDN, DNS e domainname do UTS. Se o projeto precisar de compatibilidade mais ampla no futuro, o validador pode ser parametrizado sem quebrar a semântica básica.[3][17]

5.3. Hostname como função de domínio

Uma maneira robusta de modelar a feature é tratar hostname como resultado de uma função pura de domínio, algo como `resolve_effective_hostname(requested, container_id, display_name, policy)`. O retorno dessa função não é uma string qualquer; é um value object que carrega o valor efetivo e a origem da decisão. Esse pequeno cuidado resolve vários problemas de uma vez. O `inspect` consegue explicar por que o valor foi escolhido. Restart reaplica exatamente o mesmo valor. Rename do `display_name` não contamina a identidade do guest. E o time de plataforma não fica reduzido a especular se um nome veio da CLI, de um default, de uma mutação tardia ou de um bug.

5.4. Modelo de dados

No metadata store, a solução mais limpa é separar duas agregações lógicas. A primeira é o `Container`, contendo `container_id`, `image`, `command`, `display_name` e estado lógico da workload. A segunda é o `Sandbox`, contendo `sandbox_id`, `uts_mode`, `requested_hostname`, `effective_hostname`, `hostname_source`, `domainname`, diretório de estado, `init pid`, identificadores observáveis de namespace e flags de segurança relevantes. Na primeira entrega a relação pode ser 1:1, mas o desenho já nasce correto para `exec`, `restart` e eventual compartilhamento futuro de namespace.

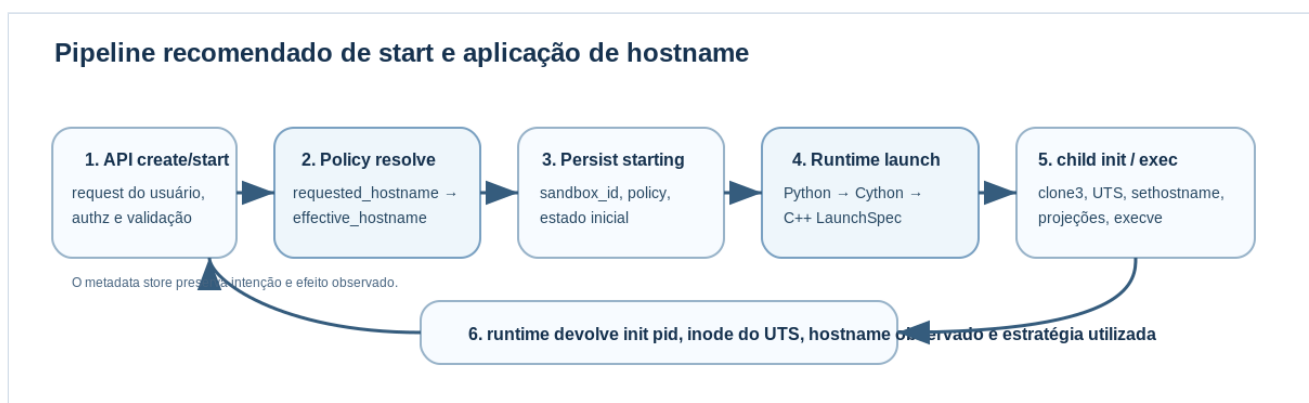
O ciclo de gravação também deve ser explícito. Primeiro, o control plane registra a intenção em estado `starting`. Depois, se a chamada ao runtime devolve sucesso, promove para `running` e persiste o estado observado. Se a materialização falha, o sandbox vai para `failed` com contexto suficiente para reconciliação e limpeza. Esse detalhe importa porque namespaces, mounts e child processes são recursos externos ao

banco; sem máquina de estados, falhas de meio de caminho deixam resíduos operacionais difíceis de explicar.

5.5. Fluxo de start

O fluxo recomendado começa na API e termina no child init nativo. A API recebe o request; o domínio valida coerência; a policy resolve o hostname efetivo; o repositório reserva IDs e grava o sandbox em **starting**. Em seguida, Python chama o runtime via Cython com o spec efetivo. O runtime escolhe a estratégia de spawn, cria o child, realiza a sincronização necessária, aplica hostname e domainname, materializa projeções de arquivo, reduz capabilities, executa a workload e devolve ao control plane o estado observado. Só então o metadado é promovido para **running**. Isso separa intenção, execução e observação, o que torna o start mais previsível e auditável.

Figura 2. Pipeline de criação e start com hostname derivado de política



5.6. Projeções de runtime: /etc/hostname e /etc/hosts

O runtime não deveria reescrever camadas da imagem para resolver a identidade do container. A estratégia mais limpa é gerar arquivos em um diretório de estado do sandbox e bind-mountá-los para dentro do rootfs. Esse desenho funciona melhor com rootfs read-only, deixa explícito que a identidade é responsabilidade do engine e evita contaminar imagens compartilhadas com dados de instância. /etc/hostname deve refletir o **effective_hostname**. Já /etc/hosts é um artefato de fronteira entre identity e networking, porque precisa conhecer hostname, aliases e endereços atribuídos.[8][17]

5.7. Semântica de uts_mode = host

O modo **host** deve ser tratado como herança, não como propriedade privada do sandbox. O control plane precisa rejeitar hostname e domainname explícitos nesse cenário, exatamente como faz o Docker. No metadata store, o valor pode ser descrito como **hostname_source = host_inherited**. O inspect não deve vender esse valor como propriedade exclusiva e imutável do sandbox, porque ele é compartilhado e, potencialmente, dinâmico.[9]

5.8. Restart, rename, clone e lifecycle

Restart deve criar novo UTS namespace quando o modo é privado, mas reaplicar o mesmo **effective_hostname** persistido anteriormente. Rederivar o valor a cada start é um erro porque muda prompt, logs, metadados observados e comportamento de agentes que dependem do hostname. Rename do **display_name** também não deveria alterar o hostname. São conceitos diferentes e pertencem a camadas

diferentes. Já operações de clone, template ou “copy as new” devem gerar novo `container_id` e, na ausência de hostname explícito, novo hostname derivado, para não multiplicar identidades idênticas acidentalmente.

5.9. Exec, attach e depuração

Se o motor quiser oferecer equivalente a `docker exec`, precisa tratar o UTS namespace como um recurso reentrante. O comando auxiliar deve aderir ao mesmo namespace do sandbox alvo antes de executar o binário solicitado. Para isso, o runtime precisa de um `NamespaceHandleRef` estável, normalmente derivado de `/proc/<init-pid>/ns/uts`, e o control plane precisa preservar o contexto necessário para a operação. É aqui que modelar o sandbox como proprietário do UTS, desde o início, paga dividendos arquiteturais.[1][4]

5.10. Rootless desde o desenho

Mesmo que o MVP seja rootful, rootless não deveria pegar o desenho de surpresa. Como o processo em um novo user namespace passa a ter capabilities naquele namespace e `sethostname()` exige `CAP_SYS_ADMIN` no user namespace associado ao UTS namespace, a sequência de boot rootless precisa ser desenhada como protocolo: criar child com users/UTS, sincronizar com o parent, aplicar mapeamentos, liberar o child e só então efetivar o hostname. O que importa aqui não é ter rootless completo no dia um; é fazer com que o formato do spec, a máquina de estados e o runtime protocol comportem esse handshake sem precisar de redesign estrutural.[3][18]

Tabela 2. Comparação entre políticas de hostname padrão

Política	Vantagens	Problemas	Recomendação
Derivar do <code>display_name</code>	Mais amigável para humanos	Renames geram deriva; caracteres podem ser inadequados; mistura control plane e guest identity	Evitar como default
Gerar palavra aleatória	UX agradável	Baixa rastreabilidade; pior reprodutibilidade; investigações mais difíceis	Usar apenas como display name opcional
Derivar de <code>container_id</code>	Determinístico, estável, técnico, fácil de auditar	Menos amigável visualmente	Melhor default
Hostname explícito do usuário	Atende contratos de aplicação e workloads legadas	Exige validação forte e ownership explícito da decisão	Suportar como override

6. Proposta de Low Level Design

6.1. Componentes do lado Python

No lado Python, a feature deve ser dividida em serviços de aplicação e value objects explícitos. Um `HostNamePolicyService` valida e deriva o hostname efetivo. Um `SandboxFactory` cria o agregado do sandbox. Um `SandboxRepository` persiste intenção e efeito observado. Um `RuntimeGateway` en-

capsula a chamada nativa. E um `InspectAssembler` monta a resposta combinando metadados, valores observados e contexto de naming. O ponto decisivo é que a lógica de negócio vive aqui; o runtime nativo não deve reinventar regras como “hostname ausente vira ID curto” ou “display name renomeável não muda o hostname”.

Os value objects principais são `ContainerId`, `DisplayName`, `Hostname`, `HostnameResolution`, `UtsPolicy` e `NamespaceHandleRef`. Em projetos apressados, tudo isso costuma virar apenas strings. O problema é que strings não carregam semântica. Um `Hostname` tem restrições e invariantes diferentes de um nome humano. Um `NamespaceHandleRef` não é um simples caminho; ele representa uma identidade observável de isolamento. Introduzir esses objetos cedo evita grande parte da confusão posterior.

6.2. Fronteira em Cython

Cython deve atuar como adaptador tipado, e não como um segundo domínio escondido. A função exposta para Python recebe um spec imutável, traduz para estruturas C++ e libera o GIL durante a chamada bloqueante ao runtime. A documentação do Cython mostra com clareza tanto o suporte a `cppclass` e `except +` quanto o uso de `with nogil` para chamadas nativas que não tocam objetos Python. Isso encaixa perfeitamente no caso de um launcher de sandbox.[15][16]

```
cdef extern from "runtime/client.hpp" namespace "engine::runtime":
    cdef cppclass LaunchSpec:
        string sandbox_id
        string container_id
        string display_name
        string requested_hostname
        string effective_hostname
        string domainname
        int uts_mode
        string existing_uts_path
        string rootfs
        string state_dir
        bint rootless

    cdef cppclass LaunchResult:
        int init_pid
        unsigned long long uts_ns_inode
        string observed_hostname
        string observed_domainname
        string spawner_kind

    LaunchResult launch(const LaunchSpec&) except +

def launch_sandbox(PyLaunchSpec spec):
    cdef LaunchSpec native = to_cpp(spec)
    cdef LaunchResult result
    with nogil:
        result = launch(native)
    return from_cpp(result)
```

O detalhe mais importante nesse snippet não é a sintaxe; é a direção da responsabilidade. O binding recebe um spec efetivo já resolvido pelo domínio e devolve um resultado estruturado. Em erros, o ideal é lançar uma exceção Python rica com `stage`, `errno`, syscall relevante e identificadores do sandbox. “Operation

not permitted” sem contexto ajuda muito pouco quando a falha pode ter ocorrido em `clone3`, `setns`, `sethostname`, `bind mount`, escrita de arquivos de projeção ou já no `execve`.

6.3. Componentes do lado C++

No lado C++, o runtime deve ser composto por peças pequenas e especializadas. Um `LaunchPlanBuilder` transforma o spec em plano de execução. Um `ProcessSpawner` escolhe entre `clone3` e `fallback`. Estratégias específicas tratam UTS: `PrivateUtsStrategy`, `HostUtsStrategy` e, quando o projeto amadurecer, `ExistingUtsStrategy`. Um `IdentityProjectionWriter` materializa `/etc/hostname` e `/etc/hosts` no diretório de estado. Um `NamespaceInspector` coleta inode/device em `/proc`. Um `CapabilityManager` executa drops e endurecimento. E um `Supervisor` usa `pidfd` e `waitid()` quando o kernel permitir.[5][6][7]

Essa composição pede wrappers RAII para descritores de arquivo, pipes, mount points e handles de namespace. Essa é uma área em que C++ bem usado vale muito a pena. Em paths com muitos estados intermediários, syscalls suscetíveis a `EINTR` e múltiplos recursos externos, vazamento de FD e limpeza incompleta são fontes clássicas de bugs. RAII reduz esse risco de forma concreta.

6.4. Sequência nativa recomendada

No caminho rootful com UTS privado, a sequência mínima recomendada é a seguinte. O parent cria pipes de sincronização e inicia o child por `clone3()` ou `fallback`. O child entra no caminho de init. Se necessário, aguarda sincronização do parent. Aplica `hostname` e `domainname`. Gera as projeções de identidade. Faz os `bind mounts` correspondentes. Aplica os drops finais de capabilities e qualquer outra política de segurança. Notifica o parent de que o ambiente está pronto. Por fim, faz `execve()` do processo final. O parent coleta pid, inspeciona o handle `/proc/<pid>/ns/uts`, registra o inode observado e devolve o resultado ao control plane.[1][3][5][6][7]

```
LaunchResult Launcher::launch(const LaunchSpec& spec) {
    SyncPipe sync;
    auto spawn = process_spawner_->spawn(spec, sync);

    if (spec.rootless) {
        userns_mapper_->write_maps(spawn.pid);
    }

    sync.signal_child_to_continue();
    auto ready = sync.wait_child_ready();

    auto ns = namespace_inspector_->inspect_uts(spawn.pid);
    return LaunchResult{
        .init_pid = spawn.pid,
        .uts_ns_inode = ns.inode,
        .observed_hostname = ready.hostname,
        .observed_domainname = ready.domainname,
        .spawner_kind = spawn.kind
    };
}

int child_init(const LaunchSpec& spec, SyncPipe& sync) {
    sync.wait_parent();
    uts_strategy_factory_.make(spec.uts_mode)->apply(spec);
    identity_projection_writer_.materialize(spec.state_dir, spec.rootfs, spec);
}
```

```

security_manager_.finalize_before_exec(spec);
sync.notify_ready(spec.effective_hostname, spec.domainname);
return execve(spec.argv[0], spec.argv.data(), spec.envp.data());
}

```

6.5. Ordem das operações e armadilhas reais

A ordem importa. Dar drop de `CAP_SYS_ADMIN` antes de `sethostname()` produz falha previsível. Escrever `/etc/hostname` sem antes garantir o valor do kernel gera inconsistência entre o que a workload lê em libc e o que enxerga por syscalls. Aplicar seccomp cedo demais pode bloquear syscalls ainda necessários ao init. Usar o processo Python do servidor para fazer `setns()` ou `unshare()` mistura camadas e torna incidentes muito mais difíceis de isolar. Esses detalhes parecem “baixo nível”, mas são exatamente o tipo de detalhe que, em produção, se manifesta como semântica quebrada de alto nível.

6.6. Compatibilidade progressiva com kernel

Nem todo ambiente terá o mesmo grau de suporte a `clone3()`, `pidfds` ou `waitid(P_PIDFD)`. Portanto, o runtime deveria descobrir capacidades do kernel no bootstrap e selecionar estratégias compatíveis, registrando no resultado qual caminho foi realmente usado. Esse metadado é valioso para troubleshooting porque ajuda a explicar por que o comportamento em desenvolvimento, CI e produção não é idêntico mesmo com o mesmo código-fonte.

6.7. Exemplo de modelagem do lado Python

```

@dataclass(frozen=True)
class UtsPolicy:
    mode: Literal["private", "host", "existing"]
    existing_path: str | None = None

@dataclass(frozen=True)
class HostnameResolution:
    requested: str | None
    effective: str | None
    source: Literal["explicit", "container_id", "host_inherited"]

@dataclass(frozen=True)
class SandboxIdentity:
    sandbox_id: str
    container_id: str
    display_name: str
    hostname: HostnameResolution
    domainname: str | None
    uts: UtsPolicy

```

Modelos assim parecem modestos, mas têm grande efeito estrutural. Eles impedem que o sistema perca a diferença entre intenção e valor efetivo, tornam a serialização para o runtime trivial e ajudam a construir respostas `inspect` honestas. Em sistemas desse tipo, a diferença entre string solta e value object não é cosmética; é governança semântica.

7. Leitura em Domain Driven Design

O problema central desta feature, sob a lente de DDD, não é o syscall. É a linguagem do domínio. Grande parte dos bugs de hostname nasce quando o domínio usa a palavra “name” para quatro coisas diferentes e deixa cada camada reinterpretar isso à sua maneira. A primeira recomendação, portanto, é construir uma linguagem ubíqua explícita e mantê-la consistente entre API, banco, logs, documentação, testes e código-fonte.

7.1. Bounded contexts

Há pelo menos três bounded contexts naturais. O primeiro é **Identity & Naming**, responsável por display name, requested hostname, effective hostname, policy de derivação e invariantes semânticos. O segundo é **Runtime Isolation**, responsável por namespaces, capabilities, launcher, exec, supervisão e integração com o kernel. O terceiro é **Networking & Discovery**, responsável por aliases de rede, DNS embutido, endereçamento e materialização de `/etc/hosts`. Misturar esses contextos numa única entidade “ContainerConfig” produz um modelo anêmico e confuso.

7.2. Aggregates, entidades e value objects

O aggregate root recomendado é o **Sandbox**. Ele possui o **UtsPolicy**, os namespace handles observados, o init pid e a identidade efetiva visível em runtime. O **Container** pode ser outro aggregate ou uma entidade fortemente associada, dependendo da amplitude do engine. Entre os value objects, vale isolar **ContainerId**, **DisplayName**, **Hostname**, **HostnameResolution**, **UtsMode** e **NamespaceHandleRef**. O ganho disso não é purismo OO; é precisão semântica.

Há um insight importante aqui: se amanhã o projeto quiser suportar mais de uma workload no mesmo sandbox, o aggregate continua válido sem refatoração conceitual. Se, ao contrário, hostname tiver sido modelado como campo do “container” desde o início, a evolução futura exigirá corrigir simultaneamente o modelo mental, a API pública e o banco de dados.

7.3. Domain services

Dois domain services se destacam. O primeiro é um **HostnameDeriver**, que calcula o valor efetivo e sua origem. O segundo é um **NamespaceAttachmentPolicy**, que resolve se o sandbox cria namespace privado, herda do host ou se liga a um handle existente. Nenhum dos dois deveria viver no runtime C++; ambos pertencem ao domínio e expressam decisões de negócio.

7.4. Domain events

Uma feature madura também se beneficia de eventos de domínio. Eventos como **SandboxIdentityResolved**, **UtsNamespaceAttached**, **HostnameApplied**, **IdentityProjectionMaterialized** e **SandboxStarted** melhoram observabilidade e auditoria sem obrigar o time a remontar a narrativa a partir de logs dispersos de syscalls. O runtime continua emitindo eventos técnicos; o control plane os traduz para a linguagem do domínio.

7.5. Repository e anti-corruption layer

O **SandboxRepository** pertence ao domínio e persiste estado de negócio. Já a tradução para um spec OCI-like ou para o protocolo do runtime deve ser tratada como uma anti-corruption layer. Isso protege o

domínio de detalhes excessivamente baixos e, ao mesmo tempo, impede que abstrações específicas de kernel vazem para a API pública sem mediação. Cython, nesse cenário, é parte dessa camada anticorrupção, não apenas um mecanismo de binding.

8. Padrões de design relevantes

Esta feature é um bom exemplo de onde padrões de design realmente ajudam sem virar ornamentação. O padrão mais útil é **Strategy**. UTS privado, host e namespace existente são estratégias distintas de attachment. Da mesma forma, hostname explícito, derivado por ID ou herdado do host também podem ser estratégias de resolução. Isso mantém o sistema aberto para extensão sem espalhar cadeias de `if` por todo o runtime e o control plane.

Builder é o segundo padrão que agrega muito valor. Um `LaunchPlanBuilder` compõe um plano efetivo a partir de múltiplas dimensões: strategy de UTS, policy de hostname, capabilities, rootless, mounts, projections e política de segurança. Em runtimes reais, construir o plano progressivamente é muito melhor do que empilhar dezenas de parâmetros num construtor gigante ou num procedimento monolítico.

Abstract Factory encaixa bem quando o runtime precisa escolher implementações dependentes do ambiente. Em kernels mais novos, a factory pode instanciar um `Clone3Spawner` com pidfd. Em ambientes mais antigos, devolve um fallback compatível. Isso encapsula heterogeneidade do kernel em um ponto controlado.

Facade aparece naturalmente no `RuntimeGateway` exposto ao Python. O control plane não deveria conhecer detalhes de pipes, pids, mount choreography, RAII guards ou supervisão nativa. Ele deveria enxergar uma fachada estável: criar sandbox, iniciar, fazer exec, inspecionar, parar. A complexidade existe, mas fica centralizada na camada correta.

Adapter é exatamente o papel da bridge em Cython. Ela adapta tipos, convenções de erro e estrutura de chamadas entre o mundo Python e o mundo C++. Sem essa leitura explícita, o binding tende a virar um lugar desorganizado onde o domínio reaparece de forma implícita e difícil de testar.

State ajuda a modelar o ciclo de vida do sandbox: `Created`, `Starting`, `Running`, `Exited`, `Failed` e, futuramente, `Deleting`. Esse padrão evita permitir operações indevidas, como `exec` antes do `init` confirmar que a identidade foi aplicada com sucesso.

Template Method também tem bom encaixe no `init` nativo. O esqueleto “preparar processo, configurar namespaces, aplicar identidade, materializar projeções, endurecer segurança, fazer exec” é estável; o que varia são passos concretos conforme rootless, modo UTS e outras features. Finalmente, **Observer** é apropriado para eventos estruturados e instrumentação, desacoplando logging, auditoria e métricas do hot path do launcher.

Fora do catálogo GoF, dois padrões também merecem destaque. **Repository** é central para persistir intenção e efeito observado. E **Specification** é excelente para encapsular regras como “hostname válido”, “combinação permitida entre `uts_mode` e hostname” e “host UTS permitido neste tenant”. Esses padrões reduzem ambiguidade numa área em que o problema é semântico antes de ser mecânico.

9. Requirements funcionais e não funcionais

Uma implementação madura dessa feature precisa ser especificada por requirements que reflitam não apenas o happy path, mas também consistência de naming, observabilidade, segurança e evolutividade. Abaixo, os requisitos são apresentados em tabelas para facilitar rastreabilidade entre produto, arquitetura e testes.

Tabela 3. Requisitos funcionais sugeridos

ID	Requisito	Comentário de design
RF-01	O engine deve criar UTS namespace privado por padrão para cada sandbox.	Base do isolamento semântico de identidade do host.
RF-02	Quando o usuário não informar hostname, o engine deve derivar um hostname efetivo determinístico a partir de identidade imutável do container e persistir o valor resultante.	Evita copiar o hostname do host por inércia e evita deriva em restart.
RF-03	O sistema deve distinguir explicitamente nome humano de control plane, hostname do sandbox, domainname do UTS e aliases de rede.	É o núcleo da consistência de naming.
RF-04	O control plane deve rejeitar combinações incoerentes, em especial hostname e domainname explícitos com <code>uts_mode = host</code> .	Reproduz a coerência semântica do Docker.
RF-05	O runtime deve aplicar o hostname no kernel via <code>sethostname()</code> na fase de init, antes do <code>execve()</code> do processo final.	Garante que a workload já nasça vendo a identidade correta.
RF-06	O engine deve projetar <code>/etc/hostname</code> e <code>/etc/hosts</code> de forma consistente com o hostname efetivo.	<code>/etc/hosts</code> depende também do bounded context de networking.
RF-07	O <code>inspect</code> deve expor <code>requested_hostname</code> , <code>effective_hostname</code> , <code>hostname_source</code> , <code>uts_mode</code> , <code>init pid</code> e identificador observável do namespace.	Sem isso, não há observabilidade honesta da feature.
RF-08	Operações equivalentes a <code>exec</code> devem aderir ao mesmo UTS namespace do sandbox alvo.	Sem isso, comandos auxiliares observam hostname diferente do processo principal.
RF-09	Restart deve reaplicar o mesmo hostname efetivo persistido para sandboxes privados; rename do display name não deve alterar hostname.	Preserva estabilidade e evita surpresas operacionais.
RF-10	O sistema deve validar hostnames contra política de caracteres e contra o limite de 64 bytes do kernel Linux.	Validação insuficiente causa falha tardia no runtime.
RF-11	O protocolo Python ↔ C++ deve transportar tanto intenção quanto efeito observado, com erros estruturados por estágio.	Essencial para suporte operacional e reconciliação.
RF-12	O design deve prever rootless e namespace compartilhado futuro, mesmo que o MVP implemente apenas o caminho rootful e privado.	Evita redesenho estrutural posterior.

Tabela 4. Requisitos não funcionais sugeridos

ID	Requisito	Critério ou orientação sugerida
RNF-01	Segurança e isolamento	CAP_SYS_ADMIN não deve ser entregue ao processo final por padrão; host UTS deve ser opt-in e policy-governed.
RNF-02	Determinismo	A derivação de hostname deve ser pura, reproduzível e persistida para não variar entre restarts.
RNF-03	Observabilidade	Eventos, logs e inspect devem carregar IDs técnicos, hostname efetivo e identificador do UTS namespace.
RNF-04	Compatibilidade progressiva com kernel	O runtime deve detectar e registrar se usou <code>clone3</code> , <code>fallback</code> , <code>pidfd</code> e outras estratégias.
RNF-05	Confiabilidade operacional	Todos os recursos intermediários devem ter cleanup robusto; wrappers RAII e state transitions explícitas são obrigatórios.
RNF-06	Baixa latência adicional	A feature de hostname deve adicionar overhead mínimo ao start; o custo deve ser pequeno frente ao bootstrap do sandbox.
RNF-07	Testabilidade	Regras de derivação devem ter testes unitários; integração deve provar isolamento observando <code>/proc/<pid>/ns/uts</code> e hostname in-container.
RNF-08	Operabilidade	Inspect e eventos devem permitir explicar por que determinado hostname foi escolhido sem exigir leitura do código-fonte.
RNF-09	Evolutividade	O modelo deve acomodar rootless, compartilhamento de namespaces, sidecars e networking mais rico sem quebrar a API principal.

10. Testes, observabilidade e operação

A implementação dessa feature deveria nascer acompanhada de uma estratégia de verificação estratificada. No nível unitário, o foco principal são as regras de naming: validação de caracteres, limite de comprimento, normalização, derivação a partir de `container_id` e rejeição de combinações inválidas de parâmetros. Esse é um caso excelente para testes baseados em propriedade porque o espaço de nomes inválidos é grande e costuma escapar de suites escritas apenas por exemplos felizes.

No nível de integração, os testes mais importantes são observacionais. Um teste deve iniciar um sandbox com UTS privado, verificar que o host real não muda, confirmar que o processo dentro do sandbox enxerga o hostname derivado e que `/etc/hostname` e `/etc/hosts` estão coerentes. Outro deve iniciar em `uts_mode = host`, provar herança do hostname do host, rejeição de configuração explícita e semântica distinta no `inspect`. Outro deve provar que um comando de `exec` entra no mesmo UTS namespace do `init` comparando os identificadores observáveis em `/proc/<pid>/ns/uts`.^{[1][2][4]}

Se rootless entrar cedo no roadmap, o conjunto mínimo de testes precisa exercitar a sincronização de user namespace, com falhas injetadas em etapas de mapeamento e observação de que o hostname só é aplicado após o child estar apto a fazê-lo. Também vale testar explicitamente o comportamento da workload quando o processo final não possui `CAP_SYS_ADMIN`: alterar hostname de dentro do container deve falhar, confirmando que a identidade ficou efetivamente congelada após o `init`.^{[3][10][18]}

No plano de observabilidade, o runtime deve emitir eventos estruturados contendo pelo menos `sandbox_id`, `container_id`, `display_name`, `effective_hostname`, `hostname_source`, `uts_mode`, `init pid`, `inode` do UTS namespace, estágio atual e erro, quando houver. Em incidentes, o par `effective_hostname + uts_ns_inode` é particularmente útil porque permite provar se dois processos enxergavam ou não a mesma identidade do kernel.

Do ponto de vista operacional, host UTS deveria ser funcionalidade conscientemente excepcional. Ele não entrega ganho expressivo de performance e reduz isolamento semântico. Sua utilidade real é compatibilidade com workloads específicas ou cenários de introspecção controlada do host. Por isso, a política prudente é torná-lo opt-in, auditado e bloqueável por tenant, profile ou ambiente.

11. Roadmap de implementação

A melhor forma de construir a feature é em fases que preservem coerência do domínio desde o início, em vez de empilhar atalhos de runtime e só mais tarde tentar “arrumar o modelo”. A seguir, o roadmap recomendado já embute essa disciplina.

Fase 1. Fundamentos de domínio e metadados

A primeira fase define linguagem ubíqua, schema de banco e contrato de API. É aqui que `container_id`, `display_name`, `requested_hostname`, `effective_hostname`, `hostname_source`, `uts_mode` e `sandbox_id` passam a existir como conceitos explícitos. Também é aqui que a política padrão de derivação e as validações de hostname são implementadas. Sem essa fase, qualquer avanço no runtime já nasce semanticamente torto.

Fase 2. MVP rootful com UTS privado

A segunda fase implementa o caminho mais valioso e mais seguro: sandbox privado rootful, hostname explícito ou derivado, projeção de `/etc/hostname`, projeção inicial de `/etc/hosts` e retorno de PID mais `inode` do namespace para o control plane. Aqui também entra o `inspect` básico e a política de drop de capabilities que torna o hostname praticamente imutável por padrão após o start.

Fase 3. Exec, restart e semântica estável

A terceira fase adiciona `exec`, `restart` e `rename` sem quebrar invariantes. É quando o investimento em `Sandbox` e `NamespaceHandleRef` começa a pagar de verdade. O objetivo é provar que processos auxiliares entram no mesmo UTS namespace e que `restart` reaplica a mesma identidade efetiva de forma determinística.

Fase 4. Host UTS, políticas administrativas e inspeção ampliada

A quarta fase adiciona `uts_mode = host`, rejeição coerente de configurações incompatíveis, marcação de `host_inherited` no metadata store e inspeção capaz de diferenciar claramente o que é herdado do host do que é propriedade privada do sandbox. Nesta fase também entram regras de autorização mais rígidas, já que host UTS é uma concessão de isolamento.

Fase 5. Rootless

A quinta fase implementa o handshake de user namespace e os testes correspondentes. Rootless não muda a semântica do domínio, mas muda bastante o low-level design do launcher. Por isso a importância de o spec e o protocolo já terem nascido capazes de expressar a sequência correta.

Fase 6. Namespace compartilhado e abstração plena de sandbox

Se o motor evoluir para sidecars, pods ou outras formas de compartilhamento de namespaces, essa é a fase em que o modo `existing` passa a existir de fato, baseado em `setns()` e handles persistidos. O custo dessa fase cai drasticamente quando hostname já tiver sido modelado desde o início como propriedade do sandbox.

12. Riscos, trade-offs e anti-patterns

O anti-pattern mais comum é usar uma única string chamada `name` para tudo. Isso parece conveniente, mas logo cria três classes de problema ao mesmo tempo: nomes humanos inválidos como hostname, renames que alteram a identidade percebida dentro do guest e conflitos entre unicidade de control plane, rede e metadados internos. A correção não é adicionar mais flags ao redor dessa string; é separar os conceitos.

Outro erro clássico é considerar que `CLONE_NEWUTS` “resolve o hostname” por si só. Não resolve. O novo UTS namespace nasce copiando os valores do namespace de origem. Se o runtime não aplicar um default ativo, a workload continuará vendo o hostname do host, apenas isolada para futuras alterações. Do ponto de vista da percepção do usuário, isso é isolamento incompleto.[2]

Um terceiro erro é usar `unshare()` ou `setns()` dentro do próprio processo Python do servidor de API. Mesmo quando legal do ponto de vista do syscall, isso é arquiteturalmente ruim. Mistura control plane e data plane, complica o modelo de thread, contamina o processo de aplicação com estados transitórios de namespace e dificulta rollback. O custo de um runtime nativo pequeno é muito menor do que o custo de depurar esse tipo de acoplamento em produção.[4][13]

Também é comum confundir o `domainname` do UTS com search domain de DNS e tentar sintetizar um FQDN automático combinando `hostname` + `domainname`. Esse atalho produz um modelo conceitual ruim porque mistura semântica de kernel, NIS e DNS. O projeto deve manter esses conceitos separados e só compor FQDN em camadas que realmente tratem de resolução de nomes.[2][8][11]

Há ainda um risco operacional importante: entregar `CAP_SYS_ADMIN` ao processo final por padrão. Se isso acontecer, a workload pode alterar seu próprio hostname em runtime e o metadata store fica exposto à deriva entre intenção, valor esperado e estado observado. Em alguns cenários privilegiados essa mutabilidade é aceitável, mas precisa ser uma escolha consciente e excepcional, não o default.[3][10]

Por fim, existe um trade-off legítimo entre compatibilidade estrita com Docker e um modelo interno mais limpo. Compatibilidade sugere aceitar semânticas e nomenclaturas historicamente consolidadas. Um modelo mais limpo sugere separar ainda mais nitidamente hostname, nodename, domainname e DNS, talvez até esconder `domainname` da API inicial. A melhor decisão, neste projeto, é equilibrar as duas coisas: manter compatibilidade mental com Docker nos defaults e no comportamento visível, mas adotar internamente um modelo semântico mais rigoroso desde o começo.

13. Conclusão

Implementar UTS namespace e hostname em um Docker-like system parece, num primeiro olhar, uma tarefa pequena: criar namespace, chamar `sethostname()` e seguir em frente. Na prática, ela é um ótimo teste de maturidade do projeto. Quem trata o tema como “mais uma flag” termina com uma API ambígua, metadados incapazes de explicar o que aconteceu e um runtime cheio de paths especiais. Quem modela identidade com rigor ganha um motor muito mais consistente.

Se fosse necessário resumir a recomendação em uma frase, ela seria esta: **modele hostname como resultado de uma política de identidade aplicada a um sandbox proprietário de UTS namespace, e não como uma string solta anexada ao container**. Essa decisão organiza o restante do sistema. Python domina o domínio e a persistência. C++ domina a fronteira com o kernel. Cython fornece um contrato tipado e eficiente. Restart, exec e rootless deixam de ser exceções improvisadas. E a semântica percebida pelo usuário permanece estável.

Essa é uma daquelas decisões arquiteturais pequenas o suficiente para caber cedo no projeto e grandes o suficiente para contaminar positivamente todo o restante do design. Quando UTS namespace é bem modelado, ele não entrega apenas isolamento de hostname. Ele entrega disciplina arquitetural.

Referências

- [1] Linux manual page namespaces(7). <https://man7.org/linux/man-pages/man7/namespaces.7.html>. Overview of Linux namespaces, /proc/<pid>/ns handles, namespace lifetime, and the use of device/inode identity for namespace equality.
- [2] Linux manual page uts_namespaces(7). https://man7.org/linux/man-pages/man7/uts_namespaces.7.html. Definition of UTS namespaces, scope of isolation, and copy-on-create behavior for hostname and NIS domainname.
- [3] Linux manual page gethostname(2) / sethostname(2). <https://man7.org/linux/man-pages/man2/gethostname.2.html>. Kernel API for gethostname/sethostname, permission model based on CAP_SYS_ADMIN in the associated user namespace, and HOST_NAME_MAX = 64 on Linux.
- [4] Linux manual page setns(2). <https://man7.org/linux/man-pages/man2/setns.2.html>. Semantics and capability checks for joining existing namespaces via file descriptors, relevant for exec and shared-namespaces designs.
- [5] Linux manual page clone(2) / clone3(). <https://man7.org/linux/man-pages/man2/clone.2.html>. Process creation API, including CLONE_NEWUTS and the structured clone3() interface used by modern runtimes.
- [6] Linux manual page pidfd_open(2). https://man7.org/linux/man-pages/man2/pidfd_open.2.html. Pidfd-based process supervision and namespace entry support.
- [7] Linux manual page waitid(2). <https://man7.org/linux/man-pages/man2/waitid.2.html>. waitid(P_PIDFD) support for robust child supervision in newer kernels.
- [8] Docker Docs - Networking. <https://docs.docker.com/engine/network/>. Default hostname behavior, per-container /etc/hosts handling, aliases, and networking interactions.
- [9] Docker Docs - docker container run. <https://docs.docker.com/reference/cli/docker/container/run/>. CLI semantics for --name, --hostname, --domainname, and --uts=host, including incompatible combinations.
- [10] Docker Docs - Runtime privilege and Linux capabilities. <https://docs.docker.com/engine/containers/run/#runtime-privilege-and-linux-capabilities>. Discussion of privileged mode and Linux capabilities; useful to explain why CAP_SYS_ADMIN should not be granted by default.
- [11] OCI Runtime Specification - config.md. <https://github.com/opencontainers/runtime-spec/blob/main/config.md>. OCI-level container configuration fields, including hostname and domainname.

- [12] OCI Runtime Specification - config-linux.md. <https://github.com/opencontainers/runtime-spec/blob/main/config-linux.md>. OCI Linux namespace model, including namespace path for joining an existing namespace.
- [13] Python 3.14 documentation - os. <https://docs.python.org/3/library/os.html>. `os.setns()` and `os.unshare()` in Python, useful to discuss prototypes and why they should not be used in the long-lived API server process.
- [14] Python 3.14 documentation - socket. <https://docs.python.org/3/library/socket.html>. `socket.sethostname()` and related hostname APIs in Python.
- [15] Cython documentation - Using C++ in Cython. https://cython.readthedocs.io/en/latest/src/userguide/wrapping_CPlusPlus.html. Typed C++ interop from Cython, including extern declarations and exception translation.
- [16] Cython documentation - Cython and the GIL. <https://cython.readthedocs.io/en/latest/src/userguide/nogil.html>. Guidance on releasing the GIL and structuring native calls from Python.
- [17] Linux manual page `hostname(5)`. <https://man7.org/linux/man-pages/man5/hostname.5.html>. Role of `/etc/hostname` as user-space configuration, distinct from the transient kernel hostname.
- [18] Linux manual page `user_namespaces(7)`. https://man7.org/linux/man-pages/man7/user_namespaces.7.html. Capabilities inside user namespaces and the foundations of rootless container bootstrapping.